HLD Level 2.10

Design a Trading / Order Processing System — Complete Interview Question & Answer

> **Interview question: Design a simplified electronic trading system that accepts buy/sell orders, validates them, matches compatible orders using price-time priority, produces executions, supports cancel/modify, publishes market data, and recovers correctly after failures. Focus on correctness, deterministic ordering, low latency, durability, backpressure and C++ implementation choices.**

1. Scope

This is a simplified exchange-style design for interview practice, not a complete production securities exchange. We focus on the order path and matching engine.

2. Functional Requirements

Accept BUY and SELL orders; support limit orders; optionally market/IOC/GFD orders; cancel orders; modify orders; maintain an order book per symbol; match compatible orders; generate trades/execution reports; expose acknowledgements/rejections; publish market-data updates.

3. Non-Functional Requirements

Very low and predictable latency, deterministic processing, strict correctness, high availability, durable recovery, no duplicate execution, bounded memory, auditable event history and horizontal scaling across symbols.

4. Core Correctness Rule

> **For a single symbol/order book, establish one deterministic order of accepted commands. Matching the same ordered input stream must produce the same output.**

5. Price-Time Priority

```
BUY side: higher price first
SELL side: lower price first

Within the same price:
earlier accepted order first
```

This is the core matching priority assumed throughout the design.

6. Example

```
SELL book:
101 -> S1 qty 10
102 -> S2 qty 20

Incoming BUY B1: price 102, qty 15

Match 10 with S1 @101
Remaining B1 qty = 5
Match 5 with S2 @102
```

After matching, S1 is removed; S2 remains with quantity 15.

7. High-Level Architecture

```
Clients
|
v
Gateway / Session Layer
|
v
Validation + Risk Checks
|
v
```

```
Sequencer / Order Router
|
v
Matching Engine Shards
|
+-> Execution Reports
+-> Market Data Events
+-> Durable Journal / Replication
```

8. Gateway Layer

The gateway authenticates clients, manages sessions, parses protocol messages, performs basic validation, assigns/request-tracks IDs and forwards valid commands. It should reject malformed input before it reaches the matching engine.

9. Example Order

```
{
order_id: 9001,
client_id: 42,
symbol: XYZ,
side: BUY,
type: LIMIT,
price: 10025,
quantity: 500,
time_in_force: GFD
}
```

10. Avoid Floating-Point Prices

Represent prices using integer ticks/minor units rather than binary floating point.

```
100.25 dollars -> 10025 cents/ticks
```

Money and matching comparisons require deterministic exact ordering. Integer price representation is the safer default.

11. Validation

Validate symbol, side, order type, positive quantity, valid tick size, price bounds, duplicate order ID and session permissions. Risk checks may validate buying power, position/credit limits and maximum order size.

12. Why Sequence Commands?

Two orders can arrive nearly simultaneously at different gateways. Wall-clock timestamps alone do not establish a safe total order. A sequencer or deterministic single-owner processing path establishes accepted order.

```
Gateway A -> order X
Gateway B -> order Y
|
v
Sequencer
|
seq 1001 = X
seq 1002 = Y
```

13. Partitioning by Symbol

```
hash(symbol) -> matching shard

AAPL -> shard 1
MSFT -> shard 3
XYZ -> shard 1
```

Different symbols can be processed independently and therefore in parallel. All commands for one symbol should reach the same logical owner at a time.

14. Why Not Parallelize One Book Aggressively?

Concurrent mutation of the same order book creates synchronization complexity and can break deterministic price-time ordering. A common design is single-threaded matching per partition/core, while scaling across symbols/partitions.

15. Single-Writer Principle

One logical matching thread owns and mutates an order book. This can eliminate locks from the matching hot path and make behavior deterministic.

16. Order Book Data Structures

A practical C++ design needs fast best-price lookup, FIFO within a price level and O(1)-style lookup for cancel/modify by order ID.

```
BUY price levels: ordered highest -> lowest
SELL price levels: ordered lowest -> highest

price -> FIFO list of orders
order_id -> iterator/reference to order
```

This directly connects to a limit-order-book implementation using ordered price levels, std::list for stable FIFO nodes, and an unordered_map from order ID to iterator/reference.

17. Matching Condition

```
Incoming BUY can match while buy_price >= best_sell_price
Incoming SELL can match while sell_price <= best_buy_price
```

18. Matching Loop

```
while incoming quantity > 0 and opposite best price crosses:
resting = oldest order at best price
trade_qty = min(incoming_qty, resting_qty)
emit trade
decrement quantities
remove fully filled resting order

if incoming remains and policy allows resting:
add to own book
```

19. Partial Fills

One order may generate multiple trades. Every execution must carry identifiers that allow downstream systems and clients to reconcile fills without ambiguity.

20. IOC

Immediate-Or-Cancel matches immediately against available liquidity. Any unfilled remainder is canceled instead of resting on the book.

21. Cancel

```
CANCEL order_id
-> lookup order_id index
-> remove from FIFO price level
-> erase order index
-> erase empty price level
```

An order_id -> iterator index avoids scanning the entire book.

22. Modify

Define semantics explicitly. A price change normally loses time priority because it is effectively cancel + new order. Quantity reduction may preserve priority depending on rules; quantity increase often loses it.

23. Acknowledgement Point

Be precise about what 'accepted' means. The system should not tell a client an order is durably accepted before reaching the durability/replication point promised by the protocol.

24. Idempotency / Duplicate Commands

Network retries can resend an order or cancel. Client/session sequence numbers and unique order IDs allow duplicate detection so a retry does not create a second order.

25. Durable Journal

Persist the ordered command/event stream so state can be reconstructed after failure.

```
seq 1001 NEW B1
seq 1002 NEW S1
seq 1003 TRADE ...
seq 1004 CANCEL ...
```

The journal is an audit and recovery mechanism. The exact production design may journal inputs, outputs or both, but replay semantics must be deterministic.

26. Snapshot + Replay

```
Periodic snapshot of order book at seq 1,000,000
+
replay journal seq 1,000,001 ... latest
```

Without snapshots, replay time grows indefinitely. Snapshot plus subsequent log replay reduces recovery time.

27. Crash Recovery

After restart, load the latest valid snapshot, replay committed events in sequence order, rebuild indexes/book state, verify the recovered sequence number, then resume processing.

28. Exactly-Once Execution

Avoid casually promising magical end-to-end exactly-once. Instead, define durable sequence IDs, idempotent consumers and reconciliation so a replayed execution event does not create a duplicate downstream effect.

29. Replication

A primary matching partition can replicate its journal/state to a standby. The failover protocol must prevent two active primaries from matching the same symbol simultaneously.

30. Split Brain

> **Two active matching engines for the same symbol can create conflicting trades. Leadership/ownership fencing is a correctness requirement, not just an availability detail.**

Use epochs/terms/leases/fencing tokens or an equivalent ownership protocol so stale leaders cannot continue committing commands.

31. Failover

```
Primary fails
-> confirm/fence old owner
-> standby verifies committed sequence
-> becomes owner with newer epoch
-> clients/router reconnect/reroute
-> processing resumes
```

32. Recovery vs Latency

Synchronous replication before acknowledgement improves durability but adds latency. Asynchronous replication lowers latency but can lose recently acknowledged work on primary failure. State the required guarantee and trade-off.

33. Backpressure

If gateways accept orders faster than a matching shard can process them, queue depth and latency grow. Never hide permanent overload behind an unbounded queue.

```
incoming 500k commands/sec
matching capacity 400k/sec
backlog grows 100k/sec
```

Apply bounded queues, admission controls, client throttling and capacity scaling. Measure queue age as well as depth.

34. Why Drop-Oldest Is Wrong Here

> **Unlike a live-camera freshness queue, an accepted trading order cannot simply be dropped because a newer order arrived.**

The overload policy depends on correctness. Reject before acceptance, throttle clients, or preserve accepted commands durably. This is an important contrast with real-time video pipelines.

35. Batching

Batching network reads, journal writes or outbound events can improve throughput, but large batches increase latency. Low-latency systems use carefully bounded micro-batching where measurements justify it.

36. Market Data

```
Matching Engine
-> book/trade events
-> market-data publisher
-> subscribers
```

Market-data publishing should not block matching. Emit ordered events into a dedicated path. Subscribers may receive snapshots plus incremental updates.

37. Snapshot + Incremental Market Data

A subscriber first receives a consistent snapshot, then applies ordered updates after the snapshot sequence. Sequence gaps trigger recovery/resynchronization.

38. Execution Reports

Clients need accepted/rejected/canceled/filled/partially-filled reports. These reports should reference order IDs, execution IDs and sequence information.

39. Slow Client

A slow client must not block the matching engine. Give each outbound session bounded buffers and an explicit policy: disconnect slow consumers, persist required reports elsewhere, or require replay/recovery through session sequencing.

40. Risk Service Dependency

Do not make a low-latency engine synchronously call a slow remote service for every matching operation unless requirements demand it. Precompute/cache limits or place deterministic pre-trade checks in the ordered path.

41. C++ Threading Model

```
Network I/O thread(s)
|
v
validated command queue
|
v
single matching thread / partition
|
+-> journal/output queues
+-> execution publisher
+-> market-data publisher
```

The matching thread should avoid blocking network, disk or logging operations directly. Use dedicated downstream paths while preserving the required durability ordering.

42. Queue Choice

Start with a bounded queue whose semantics are easy to prove. In ultra-low-latency systems, SPSC ring buffers are attractive when topology allows one producer and one consumer. Lock-free structures are useful only when they actually match the concurrency model and are measured to help.

43. Memory Allocation

Frequent heap allocation on the hot path can add latency variance. Consider preallocated object pools/ring buffers after profiling. Reserve containers where practical and keep ownership explicit.

44. Cache Locality

Compact data structures and partition affinity can improve cache behavior. Pinning a matching thread to a CPU may reduce scheduling jitter in specialized low-latency environments, but should be justified by measurement.

45. Locks

Single-writer ownership can remove book locks. Cross-thread communication then occurs through queues. This is often simpler and more deterministic than placing mutexes around every order-book operation.

46. Logging

Do not synchronously format/write verbose logs on the matching hot path. Publish compact structured events or counters to an asynchronous observability path.

47. Time

Use sequence numbers for authoritative ordering. Use monotonic time for local latency measurement. Wall-clock UTC timestamps are useful for audit/display but should not replace deterministic sequencing.

48. Latency Measurement

```
gateway receive
-> validation complete
-> sequencer
-> matching start
-> matching end
-> journal/commit point
-> response send
```

Track P50/P95/P99 and worst-case latency, not only averages. Low-latency systems care heavily about tail behavior/jitter.

49. Observability

Track commands/sec, trades/sec, rejects, cancels, queue depth/age, matching latency, end-to-end latency, journal latency, replication lag, market-data lag, session disconnects, recovery time, CPU utilization, memory, allocator activity and per-symbol hotspots.

50. Hot Symbols

Hash partitioning may still place an extremely active symbol on one overloaded shard. Since one book is difficult to split while preserving simple deterministic matching, isolate hot symbols onto dedicated cores/shards or rebalance other symbols away.

51. Rebalancing

Moving a symbol between matching shards requires a controlled handoff: stop/reroute new commands at a sequence boundary, transfer snapshot/state, establish new ownership epoch, then resume. Never allow both shards to mutate simultaneously.

52. Multi-Region

Active-active matching of the same symbol across distant regions is extremely difficult because deterministic order requires coordination. A common conceptual model is one authoritative matching location per symbol with disaster-recovery replicas elsewhere.

53. Security

Authenticate sessions, authorize accounts, encrypt network traffic where required, protect keys, validate every message length/value, rate-limit abusive clients and maintain immutable/auditable records.

54. Graceful Shutdown

```
stop accepting/routing new commands to shard
establish final sequence boundary
drain accepted command queue
flush journal/replication
publish final outputs
write snapshot/checkpoint if required
stop publishers
release resources
```

55. Complete Architecture

```
CLIENTS
|
v
Gateway / Sessions
|
v
Validation / Risk
|
v
Sequencer / Router
|
+-----------------+-----------------+
v v v
Match Shard 0 Match Shard 1 Match Shard N
symbols A,B... symbols C,D... symbols ...
| | |
+----------+------+-----------------+
|
+----------+-----------+
v v
Durable Journal Output Events
| / \
v v v
Standby/Recovery Execution Reports Market Data
```

```
|
v
Subscribers
```

56. Failure Scenario: Engine Crashes After Match

Suppose the engine calculates a trade and crashes. Correct recovery depends on the defined commit point. If the trade/command was durably committed, replay must reproduce or republish it idempotently. If it was not committed, recovery must not invent an acknowledgement that the client was promised.

57. Failure Scenario: ACK Lost

The client may retry an order whose acknowledgement was lost. Session sequence/order ID deduplication should return the previous result or reject the duplicate rather than inserting another order.

58. Failure Scenario: Journal Slow

If durability is required before acknowledgement, slow journal storage increases order latency. Queueing accepted work indefinitely is dangerous; monitor journal latency, apply backpressure/admission control and provision storage for tail latency.

59. Failure Scenario: Market Data Consumer Slow

Do not block matching. Detect lag/gaps; disconnect or resynchronize slow subscribers using a fresh snapshot plus subsequent increments.

60. Common Interview Follow-Ups

Why single-thread a book? Deterministic price-time ordering and no lock contention on the hot state.

How scale? Partition across independent symbols; isolate hot symbols.

How cancel in O(1)-style lookup? order_id -> stable iterator/reference into the price-level FIFO.

Why integer price? Exact deterministic comparison; avoid floating-point money issues.

How recover? Snapshot + ordered durable journal replay.

How avoid duplicate orders after retry? Client/session sequence and order-ID idempotency.

What if two engines own the same symbol? Prevent with fenced leadership/epoch; split brain is unacceptable.

Why not drop old orders under overload? Accepted trading commands are correctness-sensitive; throttle/reject before acceptance or preserve durably.

61. Interview-Ready Spoken Answer

"I'd partition the system by symbol and give each symbol one logical matching owner so commands for an order book are processed deterministically. Clients connect through gateways that authenticate, parse and perform basic validation and risk checks. A sequencer/router establishes the accepted command order and routes each symbol to its matching shard. Within a shard, I'd prefer a single matching thread for the owned books rather than locking the same book across many threads. The book maintains ordered buy and sell price levels, FIFO orders within each level, and an order-ID index for fast cancel/modify. Prices use integer ticks, not floating point. The matching engine emits executions and market-data events without allowing slow consumers to block matching. For durability and recovery, I'd keep an ordered journal and periodic snapshots; on restart, load the snapshot and replay subsequent committed events. A standby can replicate the journal, but failover must fence the old owner to prevent split brain. Backpressure is critical: unlike a camera pipeline, I cannot drop accepted orders to preserve freshness, so I use bounded queues, admission control and client throttling. In C++, I'd keep the hot matching path allocation- and lock-light, use explicit ownership, bounded ring buffers/queues where appropriate, asynchronous logging/output, and measure P99/worst-case latency stage by stage."

62. Connection to Your Limit Order Book Project

Your personal order-book design maps naturally to this HLD. An ordered map can maintain price priority, a list can preserve FIFO at a price level with stable iterators, and an unordered_map from order ID to iterator enables fast cancel/modify. In an interview, the important step is to zoom out from those data structures and explain sequencing, partition ownership, durability, recovery, backpressure and output isolation.

63. Five Decisions to Remember

1. Correct order matters -> SEQUENCE COMMANDS + ONE LOGICAL OWNER PER BOOK.
2. Scale without locking one book -> PARTITION ACROSS SYMBOLS.
3. Low-latency book -> PRICE LEVELS + FIFO + ORDER-ID INDEX + INTEGER PRICES.
4. Crash must not corrupt truth -> DURABLE JOURNAL + SNAPSHOT + DETERMINISTIC REPLAY.
5. Accepted orders cannot be casually dropped -> BOUNDED QUEUES + BACKPRESSURE / REJECT BEFORE ACCEPTANCE.

64. Final Revision Flow

```
Requirements
-> price-time priority
-> gateway/validation/risk
-> sequencing
-> partition by symbol
-> single-writer matching
-> order-book data structures
-> match/cancel/modify
-> journal + snapshot
-> replication/fencing/failover
-> backpressure
-> execution + market data
-> hot symbols
-> observability
-> C++ low-latency deep dive
```

65. Level 2 Completion

Level 2 complete: 1) Centralized Logging, 2) Thread Pool, 3) Camera Pipeline, 4) Telemetry Ingestion, 5) Heartbeat Monitoring, 6) OTA Updates, 7) Rate Limiter, 8) Notification System, 9) URL Shortener, 10) Trading / Order Processing System.